

Assignment 2 Report

Liam Watson

May 14, 2026

1 Introduction

This report details answers to key questions posed throughout the Digital Systems Project Assignment and analyses all waveforms given by each module using a Testbench. We aim to highlight key findings and confirm all output behaves as expected after testing.

2 Report 1

Report 1

When a module needs to be extended mid-design, several approaches are possible: modify the original in place; copy it, modify the copy, and leave the original untouched; or copy it, modify the copy, and rewrite the original as a thin wrapper around the new module. Can you think of any other options? For this specific case, which approach would you take, and how would you name the resulting module or modules? Justify your choices and discuss their trade-offs relative to the alternatives.

Figure 1: Screenshot taken from Digital_Watch_Part.2.pdf

In this instance we would prefer to create a new version of this module with a restart feature called `up_down_counter_rst`.

3 Report 2

Report 2

Do the assertions above fully capture the behaviour of an up-down counter with reset? If not, identify any behaviours left unspecified, or any properties a reasonable person might expect that the assertions do not guarantee.

Figure 2: Screenshot taken from Digital_Watch_Part.2.pdf

At this time it appears that verification may not have captured the ability to manually increment up and down nor if we wish to make active changes to **MAX** or if we wish to change the minimum value for which the counter start. Given our use case these features are not currently intended for our design.

4 Report 3

Report 3

The configuration file verifies the counter for three specific values of **MAX**. Since this does not cover all possible values, how would you formally prove correctness when this module is used in a larger design?

Figure 3: Screenshot taken from Digital_Watch_Part.2.pdf

When used in a larger design we can use proof by induction which will allow us to say with high confidence that our counter will scale for higher values of **MAX**.

5 Report 4

Report 4

Include the counterexample waveform. The general reason for this failure was explained above; use the waveform to identify the specific counterexample the checker found and explain why it is one. Then explain why restoring the assertion allows the induction step to succeed.

Figure 4: Screenshot taken from Digital_Watch_Part.2.pdf

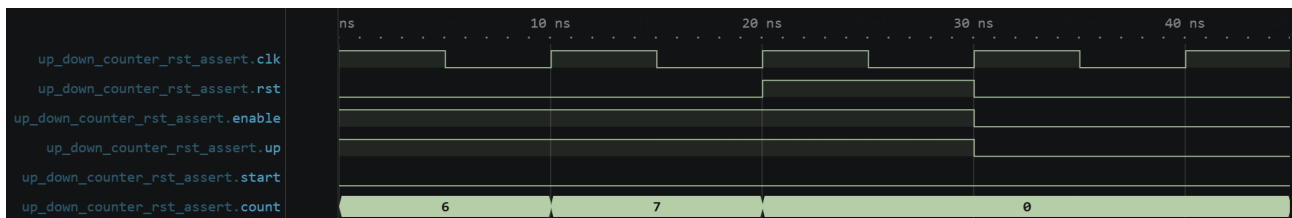


Figure 5: Measured waveform from `up_down_counter_rst_assert`

The induction check initialises count to 6. This fails as count then wraps to 0 from a value higher than **MAX** (4). Restoring the assertion then ensures that **MAX** can never exceed **MAX** meaning count can never be set above the value listed in the waveform above.

6 Report 5

Report 5

Before reading further, sketch how you would approach the design.

Figure 6: Screenshot taken from Digital_Watch_Part.2.pdf

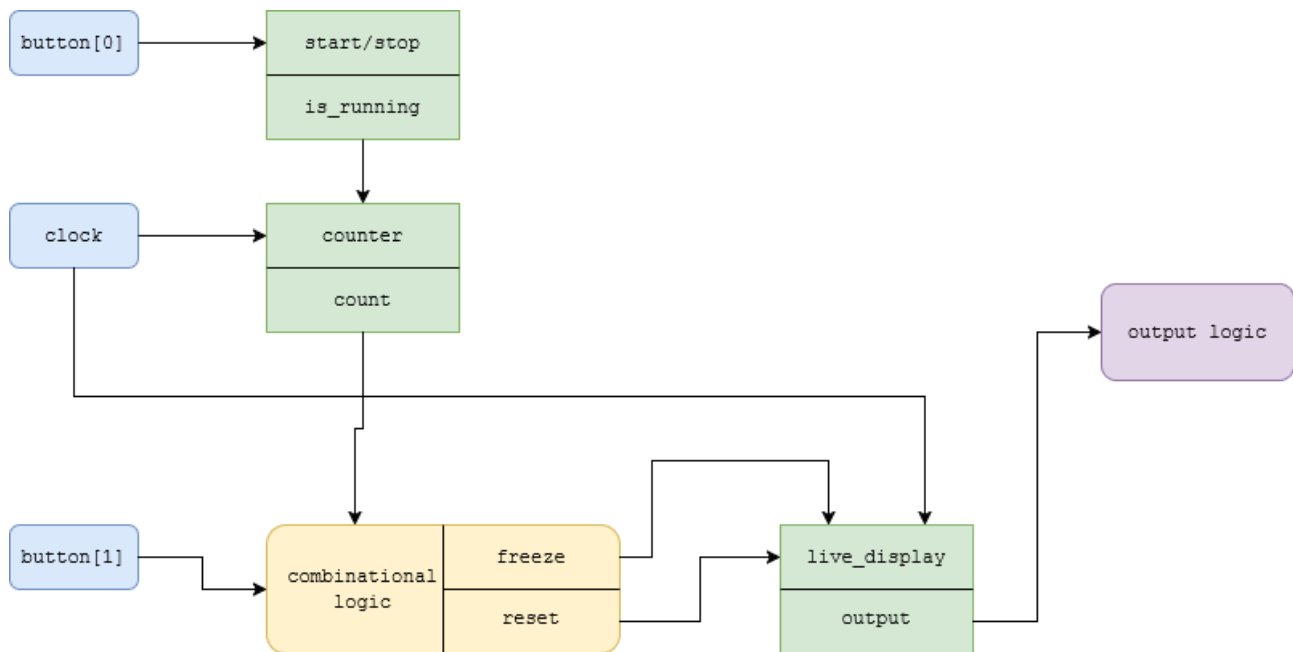


Figure 7: Sketch from diagram.io

7 Report 6

Report 6

Using a feature layering approach, how would you decompose the stopwatch into layers? Describe the responsibility of each layer.

Figure 8: Screenshot taken from Digital_Watch_Part.2.pdf

I would decompose the stopwatch into several layers. First we would handle our counter module and pass `button[0]` as an enable to control if the counter is running. I would then create a layer for the live display. I would then use combinational logic to determine the inputs to control the state for live display and use output logic to drive the stopwatch display.

8 Report 7

Report 7

Draw a block diagram of the design described above, clearly showing the data path and control signals. Compare it with your initial sketch from Section 4.2 and comment on any differences.

Figure 9: Screenshot taken from Digital_Watch_Part.2.pdf

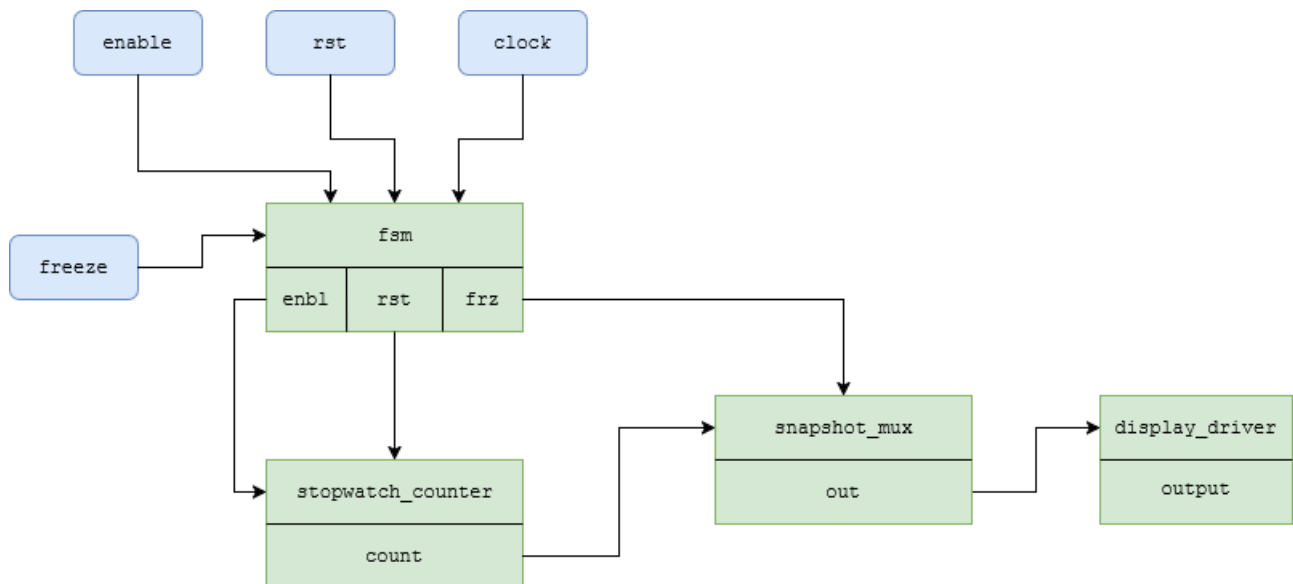


Figure 10: Block diagram created using diagram.io

9 Report 8

Report 8

Explain how your implementation satisfies the first-tick timing constraint: the first centisecond increment must occur one centisecond after **enable** goes high, not sooner.

Figure 11: Screenshot taken from Digital_Watch_Part_2.pdf

Using the **restartable_rate_generator** we generate a tick after one centisecond. This is then used alongside our enable signal to drive the **cascade_counter** module input.

10 Report 8

Report 9

Draw a diagram showing how this is implemented in hardware.

Figure 12: Screenshot taken from Digital_Watch_Part_2.pdf

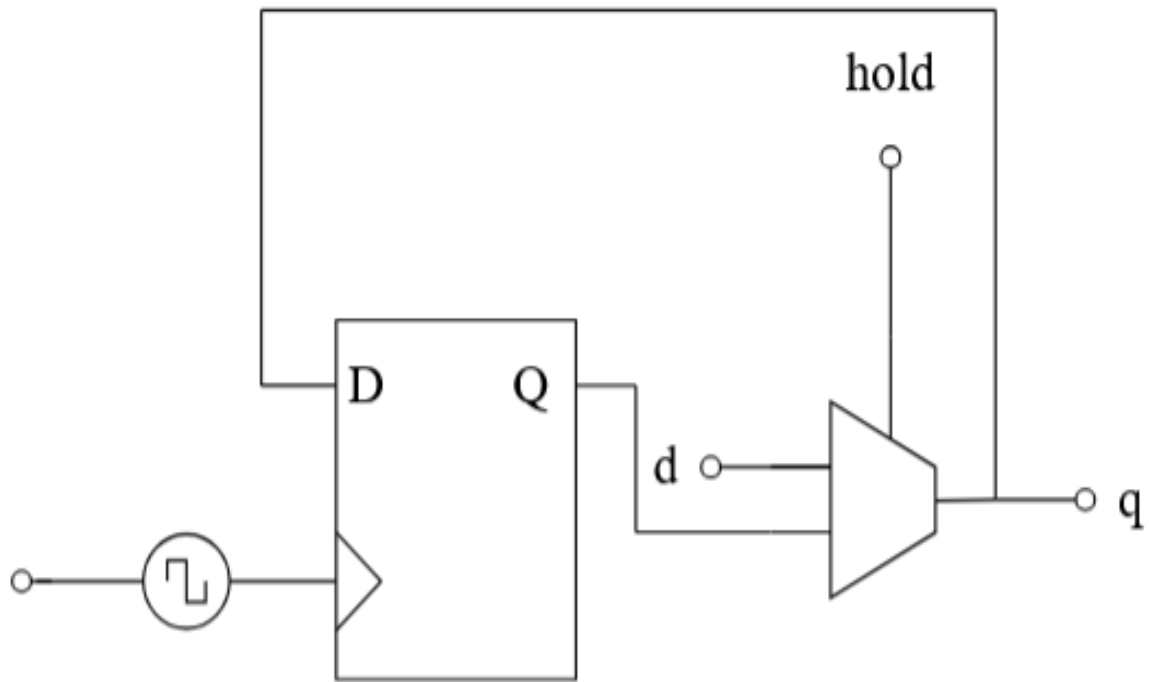


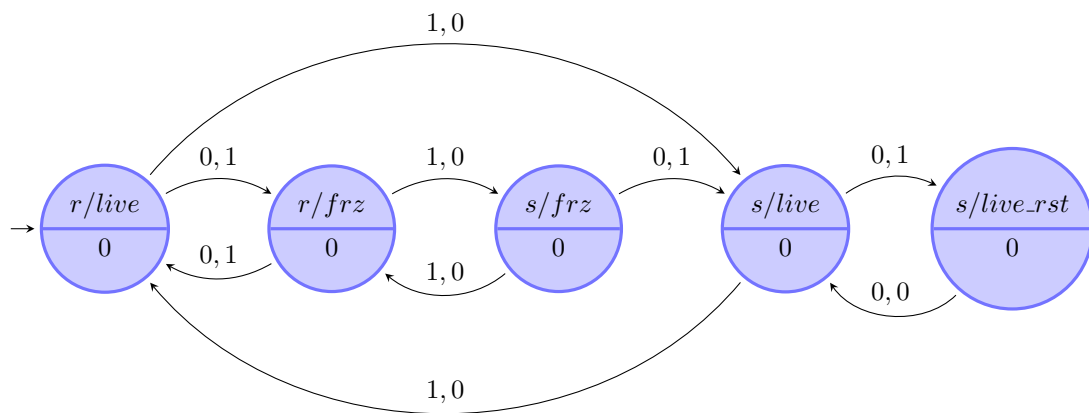
Figure 13: Created at tikzmaker.com

11 Report 10

Report 10

Draw the FSM state diagram before implementing it. After completing the implementation, note any differences between your diagram and the final design, and reflect on what caused them.

Figure 14: Screenshot taken from Digital_Watch_Part_2.pdf



12 Report 11

Report 11

Find the datasheet for the 74LS193 synchronous 4-bit up/down counter. Using the schematic diagram in the datasheet, explain how the borrow output is generated and why it behaves as it does.

Figure 15: Screenshot taken from Digital_Watch_Part_2.pdf

The borrow output is used to cascade the up and down counting functions. The borrow output produces a pulse equal in width to the count down input when the counter underflows. This allows the counter to be easily cascaded by feeding the borrow output into the count down or up inputs.

In the schematic diagram the BORROW OUTPUT comes from a 5 input NAND gate. Its first input is connected to an inverted input DOWN COUNT, and inverted outputs from Q_A , Q_B , Q_C , and Q_D . This means borrow out remains high until all inputs and COUNT DOWN are low for half negedge clock cycle.

13 Report 12

Report 12

Describe your integration plan: which features did you implement first, and why?

Figure 16: Screenshot taken from Digital_Watch_Part_2.pdf

The first feature implemented was the set mode feature. This felt the most natural and served as a basis with which to test the countdown timer features once are able to set the timer. After this was implemented Stopped Mode

14 Report 13

Report 13

The timer requires an FSM to manage its three modes. Explain your FSM design: what are the states, what are the transitions, and how did you encode the states? Justify your choices.

Figure 17: Screenshot taken from Digital_Watch_Part_2.pdf

I used a simply FSM to with to determine if the counter is running. Using a combinational block we determine whether to toggle the state between running or not running and provide a guard to prevent the state from entering into running if in edit mode or if the count is not above zero.